Shelf Management

Group 7

By

Students ID	Students Name	Section
442002988	Monerah Almobarak	81S
442003374	Nada Alotaibi	81S
442000786	Sarah Altaweel	81S
442005104	Sarah Aljuhani	81S

This project is part of Assessments Grades

for the Course Big Data

CCIS, PNU

Riyadh, KSA

Second Semester 1443 - 1444H

The Problem:

In the rapidly changing fashion industry, managing inventory effectively can be a significant challenge for retail stores. With new styles and trends constantly emerging, keeping shelves stocked with the latest and most in-demand products can be a complex task. This is particularly true in larger stores where there are many products and shelves to track and manage, as well as the risk of items becoming misplaced, which can lead to stock shortages, reduced sales, and customer dissatisfaction. The constant flow of products in and out of the store can also make it difficult to keep accurate records of stock levels and product placement, leading to inefficiencies in the inventory management process.

The Solution:

In order to overcome these challenges, a Shelf Management System for the fashion industry can be implemented. This system will automate the process of tracking inventory and ensure that shelves are always stocked with the right products. It will gather data from various sources, including barcode scanning, manual entry, IoT sensors, and stock availability data, and process this information to provide real-time insights into stock levels, item placement, and any misplaced products.

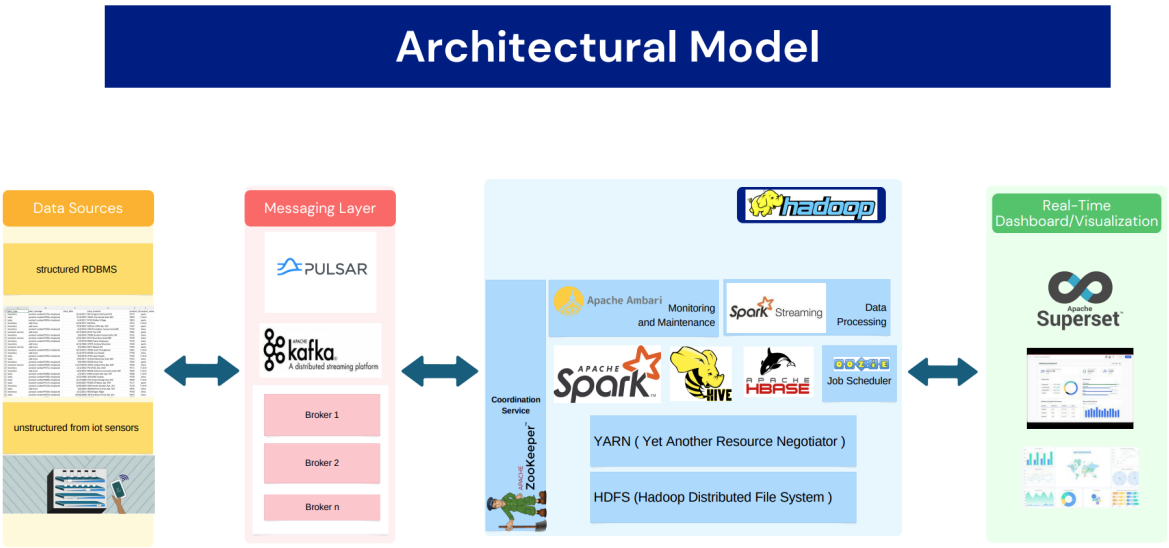
The Shelf Management System will use APIs to access sensor data from IoT sensors, such as RFID sensors, and barcode scanners. This data will be used to track the exact location and quantity of each product in stock, in real-time. The REST API will act as a communication bridge between the sensors and the Shelf Management System, allowing the system to receive and transmit data to the sensors.

For product location tracking, IoT sensors, such as RFID sensors, can be placed on each product or on the shelf to provide real-time information on the location of each item.

For product quantity tracking within the store, barcode scanners can be used to scan products as they are moved from one shelf to another, updating the stock levels in the Shelf Management System accordingly.

By integrating the sensors with the Shelf Management System, the system will be able to provide real-time insights into inventory levels, item placement, and any misplaced products. This will lead to a more efficient and streamlined inventory management process in the fashion industry, reducing the risk of stock shortages, and helping stores to meet customer demands.[1]

System architecture:



The Shelf Management System architecture would involve the following steps:[2]

- **Data Collection:** The system would collect key-value pairs such as (product ID, product_quantity_from_iot) and (product ID, product_location_from_iot) from IoT sensors, barcode scanning, and manual entry.

- **Data Ingestion:** The collected data would then be ingested into **Apache Spark** for processing.

- **Data Storage:** The processed data would be stored in scalable data storage solutions, such as **Apache HBase** for unstructured data and **Apache Hive** for structured data.

- **Data Processing:** The data would be cleansed and engineered to improve its quality, and **Spark MLlib** would be used to process the data and identify patterns and trends in stock availability and item placement.

- **Real-time Streaming:** **Apache Kafka** would be used to collect and store real-time data streams, and **Apache Spark** Streaming would be used to process the data in real-time.[3]
- **Data Analysis:** Machine learning algorithms, data mining, and statistical analysis would be used to perform deeper analysis on the processed data to uncover hidden insights and trends.[4]

- **Alerts and Notifications:** The system would send alerts and notifications to retail stores if items are running low or misplaced, using **Apache Pulsar** for real-time delivery.

- **Data Visualization:** **Apache Superset** would be used to visualize the processed data and provide insights into stock levels and item placement.

- **Security and Privacy:** The system would implement security measures such as encryption, access controls, and data masking to protect the data.

- **Disaster Recovery:** A disaster recovery plan would be in place **using HDFS replication factor 3 copies** to minimize downtime and ensure business continuity.

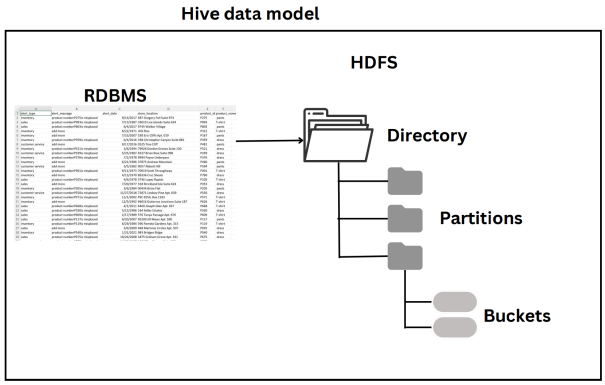
- **Monitoring and Maintenance:** The system would be regularly monitored and maintained **using Apache Ambari** to keep it running smoothly and prevent issues from arising.

Explaining the required data models:

The Shelf Management System is designed to provide a comprehensive view of products and shelves in retail stores and ensure they are fully stocked with the right products. To achieve this, the system uses two types of data models, structured and unstructured data.

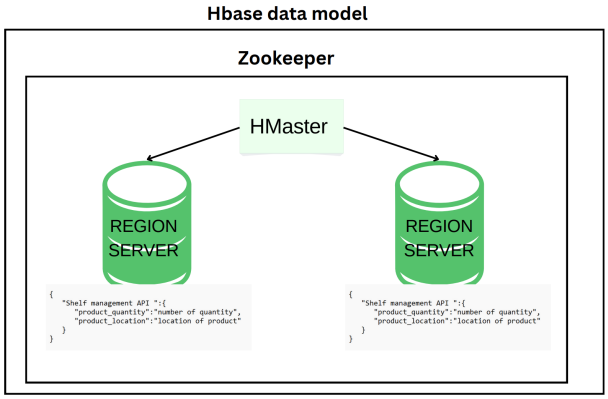
Step 1: Structured Data Storage

The structured data, including product information, shelf information, and alerts generated by the system, would be stored in Apache Hive, a data warehousing and analytics platform. This data is stored in a structured manner to allow for efficient querying and analysis.[5]



Step 2: Unstructured Data Storage

The unstructured data, such as real-time product quantity and location from IoT sensors, would be stored in Apache HBase, a distributed NoSQL database. HBase stores the real-time data from IoT sensors and provides real-time insights into stock levels and item placement.[6]



Step 3: Combining Structured and Unstructured Data

The Shelf Management System uses both structured and unstructured data to provide a comprehensive view of the products and shelves. The key-value pairs (product ID, product_quantity_from_iot) and (product ID, product_location_from_iot) are used to

track the stock availability and location of the products and to generate alerts if necessary.

Step 4: Real-Time Insights and Alerts

The combination of structured and unstructured data provides real-time insights into stock levels and item placement and ensures that the shelves are fully stocked with the right products. The Shelf Management System generates alerts for low stock levels or misplaced items using both structured and unstructured data.

Step 5: Integration with Other Systems

The Shelf Management System can also be linked to other systems, such as the Point of Sale (POS) system, for seamless integration and real-time updates on inventory and sales data.

the dataset:

The given dataset was created using the Faker library in Python to simulate an inventory and alert notifications system for a clothing store. This data can be used as a testing tool and simulation of a Shelf Management System in the clothing industry, to ensure that it is efficient and reliable in tracking stock availability, product placement, and real-time insights about stock levels. Originally, the dataset consisted of 500 rows of product data and 200 rows of alert notification data. However, due to limitations with the code, the data was manually duplicated to create a larger dataset consisting of 5000 rows of product data and 2000 rows of alert notification data. This larger dataset can provide more comprehensive insights into the effectiveness of a clothing store's inventory management system.[7]

```python
import random
import csv
from faker import Faker
fake = Faker()
used_product_ids = set()
used_shelf_ids = set()

# to generate IDs without duplicated
def generate_unique_id(used_ids, prefix):
    while True:
        new_id = prefix + str(random.randint(100, 999)).zfill(3)
        if new_id not in used_ids:
            used_ids.add(new_id)
            return new_id

# to generate product data
def generate_product_data():
    product_id = generate_unique_id(used_product_ids, 'P')
    product_name = fake.random_element(['T-shirt ', ' dress ', ' pants'])
    shelf_id = generate_unique_id(used_shelf_ids, 'S')
    product_location = fake.address()

    return {
        'product_id': product_id,
        'product_name': product_name,
        'shelf_id': shelf_id,
        'product_location': product_location,
    }

# to generate alert notification data
def generate_alert_notification_data():
    product_id = generate_unique_id(used_product_ids, 'P')
    product_name = fake.random_element(['T-shirt ', ' dress ', ' pants'])
    alert_type = fake.random_element(['inventory', 'sales', 'customer service'])
    alert_message = fake.random_element(['add more', 'product number'+ product_id+ 'is misplaced'])
    alert_date = fake.date()
    store_location = fake.address()
    return {
        'alert_type': alert_type,
        'alert_message': alert_message,
        'alert_date': alert_date,
        'store_location': store_location,
        'product_id': product_id,
        'product_name': product_name,
    }

# to save data into csv file
def save_data_to_csv(data, file_name, header):
    with open(file_name, 'w', newline='') as file:
        writer = csv.DictWriter(file, fieldnames=header)
        writer.writeheader()

        for row in data:
            writer.writerow(row)

# create 500 rows of product data
product_data = [generate_product_data() for i in range(500)]
header = ['product_id', 'product_name','shelf_id','product_location']
file_name = "product_data.csv"
save_data_to_csv(product_data, file_name, header)

#create 200 rows of alert notification data
alert_notification_data = [generate_alert_notification_data()
    for i in range(200)]
header = ['alert_type', 'alert_message', 'alert_date', 'store_location', 'product_id', 'product_name']
file_name = "alert_notification_data.csv"
save_data_to_csv(alert_notification_data, file_name, header)
```

our API JSON file format :


```
{
 "baseUrl": "https://api.shelfmanagement.com/v1",
 "endpoints": {
  "getProductLocation": {
   "url": "/products/{productId}/location",
   "method": "GET",
   "response": {
    "productId": "string",
```

```
      "location": "string"
     }
    },
    "product_location_from_iot": {
     "url": "/products/{productId}/location",
     "method": "PUT",
     "request": {
      "location": "string"
     },
     "response": {
      "productId": "string",
      "location": "string"
     }
    },
    "getProductQuantity": {
     "url": "/products/{productId}/quantity",
     "method": "GET",
     "response": {
      "productId": "string",
      "quantity": "integer"
     }
    },
    "product_quantity_from_iot": {
     "url": "/products/{productId}/quantity",
     "method": "PUT",
     "request": {
      "quantity": "integer"
     },
     "response": {
      "productId": "string",
      "quantity": "integer"
     }
    }
   }
}
```

references:

[1]“How big data can improve inventory management,” *The BigCommerce Blog*, 10-May-2022. [Online]. Available: https://www.bigcommerce.com/blog/data-inventory-management/. [Accessed: 10-Feb-2023].

[2]P. Rupareliya, “How retailers are leveraging Big Data for marketing,” *Medium*, 17-Feb-2022. [Online]. Available: https://medium.com/intuz/how-retailers-are-leveraging-big-data-for-marketing-c0d1318207c1. [Accessed: 10-Feb-2023].

[3]M. Lawson, “What is real-time inventory management and Why is it vital?,” *Digital Transformation Solutions*, 24-Dec-2020. [Online]. Available: https://www.columbusglobal.com/en-gb/blog/what-is-real-time-inventory-management-and-why-is-it-vital#:~:text=Real%2Dtime%20inventory%20management%20is,quickly%20to%20supply%20chain%20needs. [Accessed: 10-Feb-2023].

[4] S. Srivastava, “How can data analytics help improve inventory optimization in retail?,” *Appinventiv*, 05-Sep-2022. [Online]. Available: https://appinventiv.com/blog/inventory-optimization-with-data-analytics/. [Accessed: 10-Feb-2023].

[5]ligz08, “Learn-Bigdata-Udemy/hive notes.md at master · Ligz08/Learn-Bigdata-Udemy,” *GitHub*, 06-Aug-2018. [Online]. Available: https://github.com/ligz08/learn-bigdata-udemy/blob/master/Hive%20Notes.md. [Accessed: 10-Feb-2023].

[6]V. R. says: “HBase architecture: HBase Data Model: Hbase read/write,” *Edureka*, 18-Nov-2022. [Online]. Available: https://www.edureka.co/blog/hbase-architecture/. [Accessed: 10-Feb-2023].

[7]“Community providers” *Community Providers - Faker 16.8.1 documentation*. [Online]. Available: https://faker.readthedocs.io/en/master/communityproviders.html. [Accessed: 10-Feb-2023].